

Лекция. Постановка задачи отбора признаков и оценка важности признаков в линейных моделях

Задача отбора признаков состоит в том, чтобы из исходного множества признаков данных выделить такое подмножество признаков, которые будут иметь наибольшую важность для обучения и предсказания модели. Отбор признаков проводится по следующим причинам:

- Сокращение времени обучения. Многие наборы данных содержат кучу информации об объектах, при логировании различных систем в базы данных записываются гигабайты информации. Однако, информация, необходимая, например, для какого-то ведомства, может оказаться совершенно не нужной при построении модели машинного обучения.
- Повышение качества моделей. Нерелевантные признаки могут ухудшать точность моделей, поскольку будут играть роль шума.
- Борьба с переобучением (и в целом улучшение обобщающей способности модели). Удаление некоторых признаков может повысить обобщающую способность моделей.
- Повышение интерпретируемости моделей. Если признаков меньше, решение модели легче интерпретировать. К тому же упрощается процесс визуализации.
- Сокращение памяти, которую занимает модель. Поскольку размер входных данных уменьшается, то уменьшается и размер итоговой модели. Стоит отметить, что отбор признаков не является извлечением признаков. Извлечение признаков преобразует исходное представление объекта для получения нового небольшого набора признаков, в то время, как снижение размерности – это один из методов извлечения признаков.

Линейная регрессия. Рассмотрим модель линейной регрессии:

$$f_{linreg}(x) = \sum_{i=1}^d w_i x_i + \lambda ||w||_*, \quad \text{где } x = (x_1, \dots, x_d) \text{ — объекты выборки, } w = (w_1, \dots, w_d) \text{ —}$$

веса модели $a_{linreg}(x)$, λ — коэффициент регуляризации, $||w||_*$ — норма весов. Таким образом, каждому признаку соответствует какое-то значение веса. Если выборка нормализована, то получившиеся веса признаков можно трактовать как важность признаков и наиболее важным является признак с наибольшим весом. Если выборка не нормализована, то в качестве важности признаков можно использовать p-значения (p-values), соответствующие коэффициентам линейной регрессии: сначала устанавливается уровень значимости, а затем проверяются полученные p-значения – если какое-то значение оказывается ниже выбранного нами уровня значимости, то данный признак объявляется значимым, т.е. вероятно, что изменение его значения приведет к изменению предсказания целевой переменной.

Аналогичным образом можно трактовать и **веса логистической регрессии**, решающей задачу классификации, поскольку она представляет собой ту же линейную комбинацию векторов весов и признаков, однако, поверх этого берется нелинейная функция активации. Более подробно оценка важности признаков в линейных моделях и ансамблях решающих деревьев будут разобраны в следующей теме.

Оценка значимости признаков для линейных моделей и для ансамблей решающих деревьев

Рассмотрим подробнее оценку значимости признаков в линейных моделях и ансамблях решающих деревьев. Для них подходят упоминавшиеся ранее методы и несколько особенных:

- Для линейных моделей: если признаки нормализованы, в качестве важности признаков можно использовать веса модели. Чем больше вес модели, тем более важным является признак
- Для ансамблей решающих деревьев признак тем более важен, чем чаще он встречается в ветвлениях

Лассо регрессия.

Для начала в качестве линейной модели более подробно рассмотрим модель линейной регрессии с L_1 - регуляризацией (можно рассматривать и модель с L_2 - регуляризацией, но по эмпирическим наблюдениям Лассо работает лучше гребневой регрессии. Визуальные различия между Лассо и гребневой регрессией приведены на Рис.1.

$$f_{linreg}(x) = \sum_{i=1}^d w_i x_i + \lambda ||w||_1, \text{ где } x = (x_1, \dots, x_d) \text{ — объекты выборки, } w = (w_1, \dots, w_d) \text{ — веса модели } a_{linreg}(x), \lambda (\lambda \geq 0) \text{ — коэффициент регуляризации, } ||w||_1 - L_1 \text{ норма весов. Оптимизационная задача будет выглядеть следующим образом:}$$

$$\min_w F(w) = \lambda ||w||_1 + \sum_{i=1}^d (w x_i - y_i)^2, \text{ где } y_i \text{ — правильные ответы.}$$

Решение данной задачи будет эквивалентно выпуклому квадратичному программированию (QP), поэтому можно пользоваться как стандартными решениями QP, так и специальными алгоритмами, например LARS (регрессия на основе наименьших углов). Преимуществом такого алгоритма является предоставление строгих теоретических гарантий решения, однако данный метод имеет и ряд недостатков:

- Нет возможности использовать ядра
 - Нет решения в явной форме (не обязательно, но могло бы быть удобно для теоретического анализа)
 - Если есть группа переменных с большой попарной корреляцией, то Лассо будет выбирать только одну переменную из группы
- Эмпирическое правило для улучшения качества предсказания: сначала необходимо сделать отбор признаков с помощью Лассо, а затем оценить параметры модели заново с помощью гребневой регрессии, используя отобранные признаки.

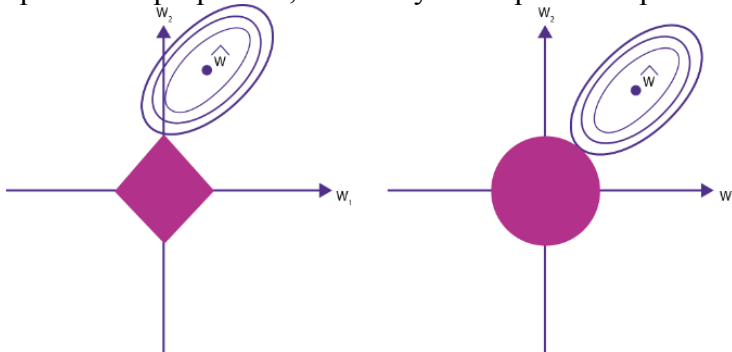


Рис. 1. Сравнение Лассо (слева) и

гребневой (справа) регрессий, пример для двумерного пространства независимых переменных. Фиолетовые области изображают ограничения на коэффициенты, эллипсы — некоторые значения функции наименьшей квадратичной ошибки

Для того чтобы исправить один из недостатков регрессии Лассо, была введена модель **Elastic Net**, которая содержит в себе сразу два вида регуляризации: L_1 и L_2 . Оптимизационная задача для нее выглядит следующим

образом:

$$\min_w F(w) = \lambda((1 - \alpha)||w||_1 + \alpha||w||_2^2) + \sum_{i=1}^d (wx_i - y_i)^2,$$
 где α ($\alpha \in [0, 1]$) — вес выпуклой комбинации.

Elastic Net позволяет проводить отбор признаков с возможностью выбора группы признаков с большой попарной корреляцией.

Ансамбли решающих деревьев

Для начала рассмотрим пример конкретного дерева из ансамбля (Рис. 2).

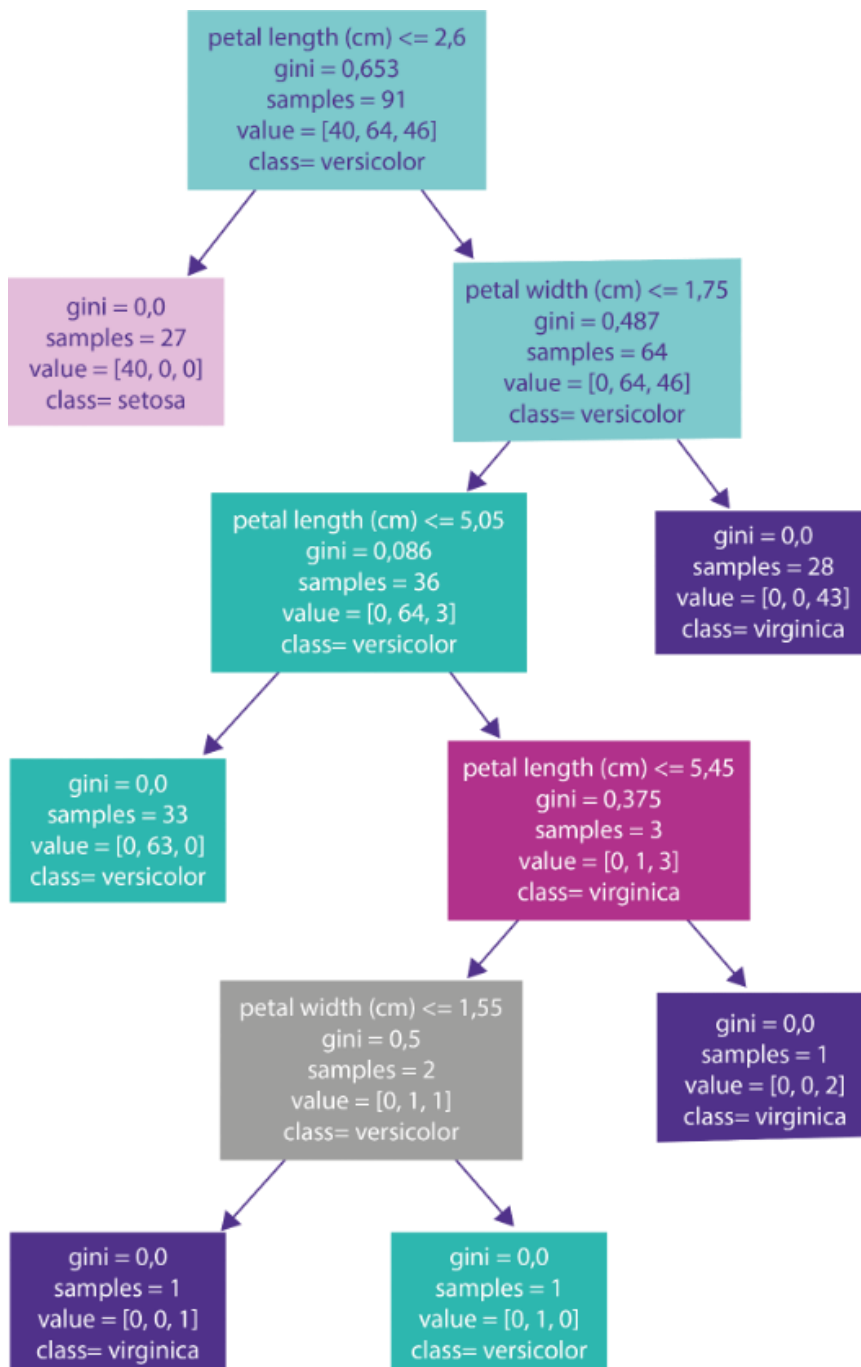


Рис. 2. Пример обученного решающего дерева из ансамбля

Решающее дерево на Рис. 2 — одно из 100 решающих деревьев случайного леса, обученного для задачи классификации ирисов. В рассматриваемом дереве встречаются следующие признаки:

- petal length (cm) — упоминается 3 раза
- petal width (cm) — 2 раза

Таким образом, для данного конкретного дерева наиболее важным является признак *petal* *length* (cm).

Поскольку в ансамбле содержится не одно дерево, можно получить количество вхождений в узлы каждого признака для каждого дерева, а затем агрегировать полученные значения. Однако данный метод редко применяется на практике из-за того, что он отдает больше важности числовым признакам, чем бинарным или признакам с малым количеством категорий. Чтобы убрать данный недостаток, был введен следующий алгоритм оценки важности признаков (permutation feature importance):

1. Модель обучается на обучающей выборке
2. Качество модели оценивается на валидационной выборке D . Обозначим данное качество Q
3. Для каждого признака i валидационной выборки D создаем новую валидационную выборку D'_i , в которой значения признака i перемешиваются. Таким образом сохраняется распределение признака, но теряется его смысл. Для новой выборки D'_i получаем качество модели Q'_i . Значение $Q - Q'_i$ и будет важностью признака i : чем выше это значение, тем более важен признак i для модели

Однако и этот метод имеет свои недостатки:

- Если признаки скоррелированы, важность будет разделена между ними, из-за чего важный признак может оказаться неважным
- Если в выборке имеются категориальные признаки, закодированные с помощью one hot encoding, важность будет делиться между значениями категорий

Таким образом, выбор метода оценки важности признаков зависит от конкретной задачи и ее контекста. Если выборка состоит только из численных признаков, можно использовать оба метода для ансамблей решающих деревьев. Если выборка все-таки содержит и категориальные признаки, то стоит обратить внимание на permutation feature importance.

Использование значений SHAP для оценки значимости

В предыдущих лонгридах были разобраны алгоритмы оценки значимости признаков для линейных моделей и ансамблей решающих деревьев. Однако рассмотренные ранее техники оценивания значимости и встроенные в различные пакеты, такие как sklearn, xgboost и т. д., не дают представления о том, как именно влияет тот или иной признак на предсказание модели. Для того чтобы все-таки узнать, как значения того или иного признака влияют на решение модели, было предложено использовать **shap-значения**.

Подход **SHAP (Shapley additive explanation values)** позволяет оценивать важность признаков в произвольных моделях машинного обучения. Изначально понятие **значений Шепли (Shapley values)** было введено в теории игр. Значения Шепли позволяют численно оценивать вклад каждого игрока при игре в кооперативную игру (игра, в которой группа игроков (*коалиция*) действует совместно) для достижения общей цели.

Определение значений Шепли. Пусть существует характеристическая функция v , которая каждому множеству игроков сопоставляет число — эффективность данной коалиции игроков, действующей совместно. Тогда значение Шепли для каждого игрока — это число, рассчитываемое по формуле:

$$\Delta(i, S) = v(S \cup i) - v(S), \quad (1)$$

где $\Delta(i, S)$ — прирост эффективности от добавления игрока i в коалицию S .

Пусть всего N игроков. Рассмотрим множество Π всех возможных упорядочиваний игроков и обозначим за $p_{i\pi}$ множество игроков, стоящих перед игроком i в упорядочивании π . Тогда значение Шепли для игрока рассчитывается следующим образом:

$$\Phi(i) = \frac{1}{N!} \sum_{\pi \in \Pi} \Delta(i, p_{i\pi}), \quad (2)$$

т. е. мы считаем средний прирост эффективности от добавления игрока i в коалицию игроков, стоящих перед ним, по всем возможным упорядочиваниям игроков.

Формула (2) задается аксиоматически, т. е. формулируется ряд необходимых свойств и доказывается, что данное решение является единственным, которое им удовлетворяет. Поскольку $\Delta(i, S)$ не зависит от порядка игроков в S , можно объединить равные друг другу слагаемые и переписать формулу

$$\Phi(i) = \sum_{S \subseteq \{1, 2, \dots, N\} \setminus i} \frac{|S|! (|N| - |S| - 1)!}{N!} \Delta(i, S). \quad (3)$$

в эквивалентном виде:

Формула (3) является взвешенной суммой по всем подмножествам игроков, не содержащих игрока i , в которой веса принимают наибольшие значения при $|S| \approx 0$ или $|S| \approx |N|$ и наименьшие значения при $|S| \approx |N| / 2$.

Значения Шепли можно применять в машинном обучении, если игроками считать наличие отдельных признаков, а результатом игры — ответ модели на конкретном примере x .

Значения регрессии Шепли (Shapley regression values) позволяют оценить вклад каждого признака в ответ модели f . Зафиксируем конкретный пример x из тестовой выборки и обучающую выборку. За характеристическую функцию множества признаков будем считать предсказание модели, обученной только на этих признаках: $v(S) = f_S(x_S)$. (4)

Следовательно, $\Delta(i, S)$ — изменение в предсказании x между моделью $f_{S \cup \{i\}}$, обученной на признаках $S \cup \{i\}$, и моделью f_S , обученной на признаках S :

$$\Delta(i, S) = f_{S \cup \{i\}}(x_{S \cup \{i\}}) - f_S(x_S). \quad (5)$$

Тогда вклад отдельных признаков в величину предсказания модели можно оценивать по формулам (2) и (3). Отметим, что мы рассматриваем не вклад

каждого признака в точность модели, а вклад каждого признака в *величину предсказания модели на конкретном тестовом примере*, что помогает интерпретировать это предсказание. Если модель предназначена для классификации, то в выходными значениями считаются logits – значения до нелинейной операции (в качестве которой наиболее часто выступают softmax или sigmoid). Но можно рассматривать и вклад признаков в величину метрики качества (SHAP loss values).

1. Для игрока существует всего два значения: он либо есть, либо его нет, в то время как признак, как минимум, принимает два разных значения или отсутствует. В значениях регрессии Шепли сравнивается текущее значение признака на примере x с его полным отсутствием при обучении и тестировании.

2. Высокая сложность вычислений. Для расчета значений регрессии Шепли нужно обучать модель на всех возможных подмножествах признаков, что в большинстве случаев невыполнимо. Однако мы можем намного быстрее посчитать приблизительное значение Shapley values, если будем считать не все элементы суммы (3), а лишь некоторые, обладающие большими весами. Для этого мы можем использовать веса как вероятности при семплировании элементов суммы (аппроксимация семплированием, *Shapley sampling values*, <https://dl.acm.org/doi/10.1007/s10115-013-0679-x>).

Применим статистический подход и будем считать, что обучающие и тестовые данные взяты из некоторого распределения вероятностей. Пусть часть признаков в примере x известны, часть пропущены. Обозначим за x_s известные признаки. В SHAP характеристическая функция множества признаков S для примера x и модели f задается как условное мат. ожидание: $v(S) = E[f(x)|x_s]$. (6)

Формула (6) означает, что за $v(S)$ мы берем мат.ожидание предсказания f на примерах x' , взятых из распределения данных, таких, что $x_{s'} = x_s$. Тогда **определение значений Шепли** выглядит следующим образом: даны модель f , распределение данных и некий тестовый пример x . Задача состоит в оценке важности текущих значений каждого признака по сравнению с их неопределенными значениями. SHAP (SHapley Additive exPlanation)-значения для признаков на примере x – это значения Шепли, рассчитываемые для следующей кооперативной игры:

- Игроками являются признаки: наличие $i^{\text{ого}}$ игрока означает текущее значение $i^{\text{ого}}$ признака на примере x ; отсутствие $i^{\text{ого}}$ игрока означает неопределенное значение $i^{\text{ого}}$ признака.
- Характеристической функцией $v(S)$ коалиции признаков S является условное мат.ожидание $E[f(x)|x_s]$ по распределению данных.

Таким образом, алгоритм расчета SHAP-значений по формулам (2) и (3):

- 1) для каждого возможного упорядочивания признаков берутся все признаки, стоящие перед $i^{\text{ым}}$ признаком (обозначим их как S);
- 2) считается величина $\Delta_f(i, S) = E[f(x)|x_{S \cup i}] - E[f(x)|x_S]$;
- 3) полученные значения усредняются по всем упорядочиваниям.

Т.е. значения SHAP описывают *ожидаемый прирост выходного значения модели при добавлении $i^{\text{ого}}$ признака в текущем примере.*

Наиболее важным свойством, которым обладают значения Шепли, рассчитанные по формулам (2) и (3) является *состоятельность (consistency)*: если (зафиксировав пример x) мы рассчитываем значения Шепли для двух моделей f и f' , и для любых S , i верно, что вклад признака i при имеющемся множестве признаков S в первой модели не меньше, чем во второй, то значения Шепли этого признака в первой модели также не меньше, чем во второй:

$\forall S, i: \Delta_f(i, S) \geq \Delta_{f'}(i, S) \Rightarrow \phi_f(x) \geq \phi_{f'}(x).$

Отличие значений Шепли от значений регрессии Шепли в том, что в последних характеристической функцией группы признаков x_S является значение $f_{x_S}(x_S)$, а в значениях Шепли – $E[f(x)|x_S]$. В целом эти значения близки, так как f_{x_S} , как правило, моделирует $E[y|x_S]$. Но значения регрессии Шепли требуют многократного обучения модели и, таким образом, являются характеристикой обучаемой модели, тогда как значения Шепли – характеристикой обученной модели.

Однако, значения Шепли имеют некоторые недостатки:

- Вычислительные сложности. Определение SHAP не говорит о том, как именно считать $E[f(x)|x_S]$ в условиях ограниченной выборки.
- Ограниченная применимость. Значения Шепли не являются универсальным способом интерпретации потому, что выходные данные могут иметь сложный формат, а входные признаки могут быть не интерпретируемы.
- Сложная концепция. При изучении данного метода могут периодически возникать вопросы, а действительно ли значения Шепли можно считать мерой важности признаков в модели? Такой вопрос возникает из-за того, что SHAP-значения зависят не только от модели, но и от распределения данных, при этом даже признаки, которые никак не используются моделью, могут иметь ненулевые значения Шепли. Однако, для устранения некоторых из этих недостатков можно использовать модификации SHAP, например, Independent SHAP (приблизительно рассчитывает сумму (3) с помощью предположения о том, что наличие каждого признака линейно и независимо влияет на ответ модели), Kernel SHAP (приблизительно рассчитывает сумму (3) с помощью подсчета не всех, а лишь части ее элементов), Tree SHAP (работает с моделями, основанными на решающих деревьях) и т.д.

Для подсчета значений Шепли на практике можно использовать библиотеку `shap`, которая содержит не только вычисление значений Шепли, но и множество методов визуализации как самой важности признаков, так и влияния значений каждого признака на предсказание. Подробнее про практическое применение библиотеки `shap` можно почитать в [**jupyter notebook**](#).

Задача отбора признаков

Задача отбора признаков состоит в том, чтобы из исходного множества признаков данных выделить такое подмножество признаков, которые будут иметь наибольшую важность для обучения и предсказания модели.

Методы отбора признаков делятся на две большие категории:

- Методы, не зависящие от обучающего алгоритма. Среди них выделяют:

- Методы, основанные на предобработке. Например, удаление бинарного признака, если он принимает одно из двух значений только на небольшом числе примеров.

- Методы, основанные на фильтрации: сначала признаки ранжируются в соответствии с некоторым критерием, затем выбираются несколько первых признаков из них. Для ранжирования используют эвристики, которые, однако, обычно быстро работают. Наиболее популярными такими эвристиками являются:

- Корреляция: признаки ранжируются по их корреляции с выходной

переменной:
$$r(x_j, y) = \frac{cov(x_j, y)}{\sqrt{var(x_j)var(y)}},$$
 где $var(\cdot)$ – дисперсия, $cov(\cdot, \cdot)$ – ковариация.

$$MI(x_j, y) = \sum_{x_j} \sum_y p(x_j, y) \log \frac{p(x_j, y)}{p(x_j)p(y)}.$$

- Взаимная информация:

Большая взаимная информация означает высокую значимость признака. Вероятности внутри формул оценивают по данным.

Однако, данные методы не учитывают взаимосвязи между признаками.

- Можно также оценивать важность признаков следующим образом:

- В линейной регрессии с нормированными признаками наиболее важным является признак с наибольшим весом.

- В градиентном бустинге наиболее важный признак – тот, который чаще всего встречается в ветвлениях.

- Надстройка над обучающим алгоритмом. Алгоритм обучения запускается несколько раз с разными наборами признаков. Данная технология может быть вычислительно затратной.

- Прямой проход (Add). Изначально имеется пустое множество, в которое жадно включаются наиболее релевантные признаки и оценивается качество модели с новым набором признаков. Остановка происходит при достижении желаемого количества признаков.

- Обратный проход (Del). Изначальное множество включает в себя все признаки. Далее жадно удаляют наименее релевантные признаки и оценивается качество модели с новым набором признаков. Остановка происходит при достижении желаемого количества признаков.

- Комбинация прямого и обратного прохода (AddDel). Добавление и отбрасывание признаков делаются последовательно.

Данные методы требуют аккуратной оценки их реального эффекта, т.е. необходимо делать правильную перекрестную проверку (кросс-валидацию).

- Встроенные методы. Отбор признаков происходит при построении модели.

Рекомендации по отбору признаков:

1. Сначала необходимо с помощью методов, основанных на предобработке и фильтрации убрать ненужные признаки, такие как константные признаки, различные идентификаторы, пустые признаки и т.д.

2. Далее, с помощью более сложных алгоритмов, например, надстроек над обучающим алгоритмом или с помощью встроенных методов, проводится отбор признаков.